

TensorFlow 2.0 and Keras

TensorFlow (TF) is a specialized numerical computation library for deep learning. It is the preferred tool by numerous deep learning researchers and industry practitioners for developing deep learning models and architectures as well as for serving learned models into production servers and software products. This chapter is focused on TensorFlow 2.0.

Navigating Through the TensorFlow API

Understanding the different levels of the TF API hierarchy is critical to working effectively with TF. The task of building a TF deep learning model may be addressed via different TF API levels. An understanding of the API hierarchy provides clarity on implementing neural network models with TF as well as navigating the TF ecosystem. The TF API hierarchy is primarily composed of three API levels, the high-level API, the mid-level API which provides components for building neural network models, and the low-level API. A diagrammatic representation of this is shown in Figure 30-1.

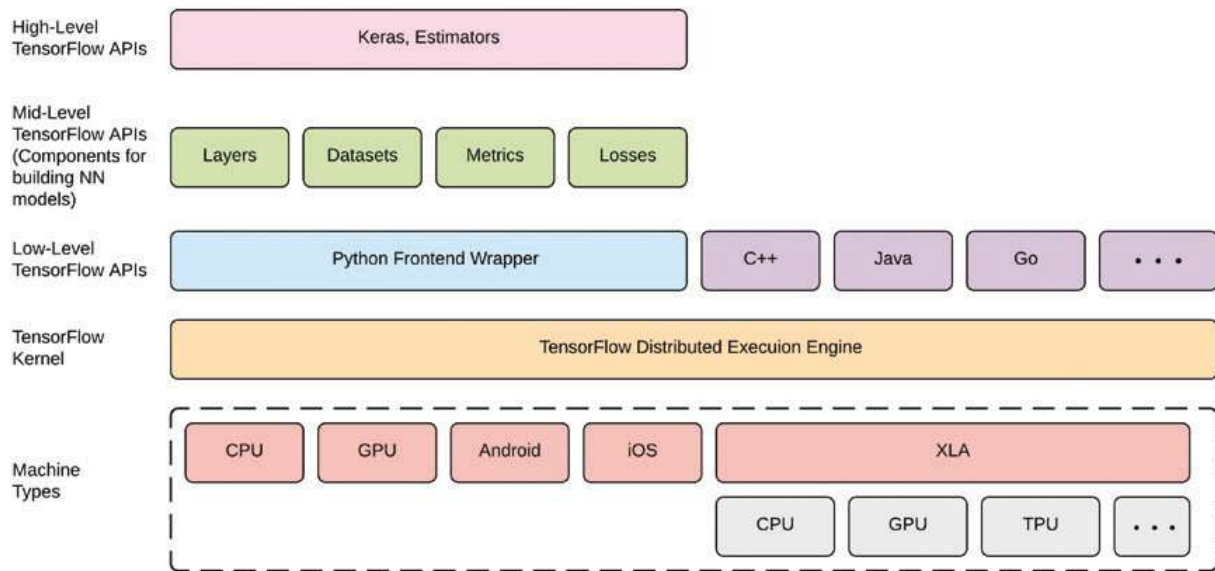


Figure 30-1. TensorFlow API hierarchy

The Low-Level TensorFlow APIs

The low-level API gives the tools for building network graphs from the ground up using mathematical operations. This API level affords the greatest level of flexibility to tweak and tune the model as desired. Moreover, the higher-level APIs implement low-level operations under the hood.

The Mid-Level TensorFlow APIs

TensorFlow provides a set of reusable packages for simplifying the process involved in creating neural network models. Some examples of these functions include the layers (**`tf.keras.layers`**), Datasets (**`tf.data`**), metrics (**`tf.keras.metrics`**), loss (**`tf.keras.losses`**), and FeatureColumns (**`tf.feature_column`**) packages.

Layers

The layers package (**`tf.keras.layers`**) provides a handy set of functions to simplify the construction of layers in a neural network architecture. For example, consider the convolutional network architecture in Figure 30-2 and how the layers API simplifies the creation of the network layers.

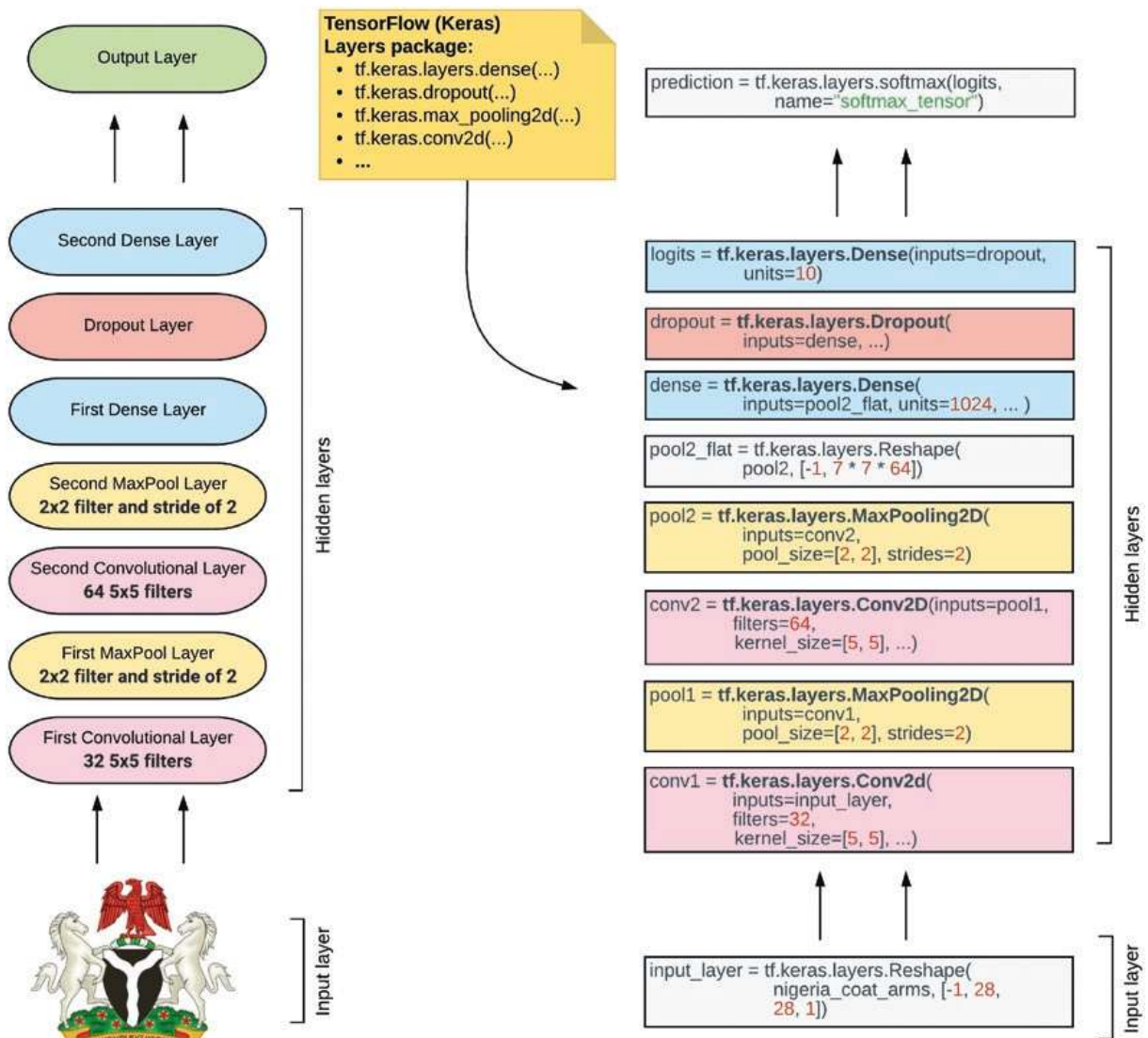


Figure 30-2. Using the layers API to simplify creating the layers of a neural network

Datasets

The Dataset package (**tf.data**) provides a convenient set of high-level functions for creating complex dataset input pipelines. The goal of the Dataset package is to have a fast, flexible, and easy-to-use interface for fetching data from various data sources, performing data transform operations on them before passing them as inputs to the learning model. The Dataset API provides a more efficient means of fetching records from a dataset. The major classes of the Dataset API are illustrated in Figure 30-3.

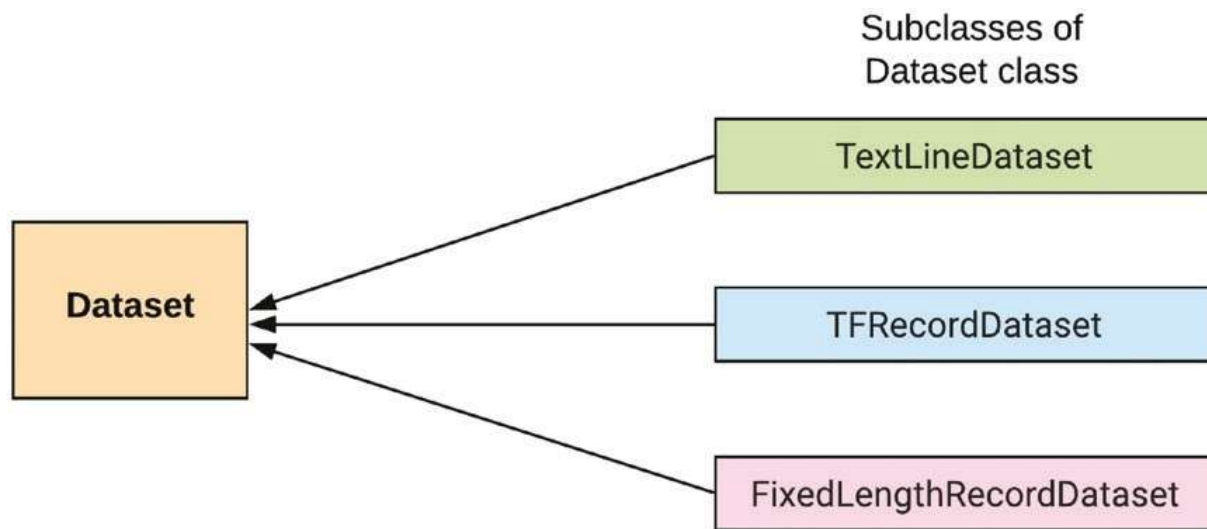


Figure 30-3. *Dataset API class hierarchy*

From the illustration in Figure 30-3, the subclasses perform the following functions:

- **TextLineDataset:** This class is used for reading lines from text files.
- **TFRecordDataset:** This class is responsible for reading records from TFRecord files. A TFRecord file is a TensorFlow binary storage format. It is faster and easier to work with data stored as TFRecord files as opposed to raw data files. Working with TFRecord also makes the data input pipeline more easily aligned for applying vital transformations such as shuffling and returning data in batches.
- **FixedLengthRecordDataset:** This class is responsible for reading records of fixed sizes from binary files.

FeatureColumns

FeatureColumns **tf.feature_column** is a TensorFlow functionality for describing the features of the dataset that will be fed into a high-level Keras or Estimator models for training and validation. FeatureColumns makes it easy to prepare data for modeling by carrying out tasks such as the conversion of categorical features of the dataset into a one-hot encoded vector.

The **feature_column** API is broadly divided into two categories; they are the categorical and dense columns. The categories and subsequent functions are illustrated in Figure 30-4.

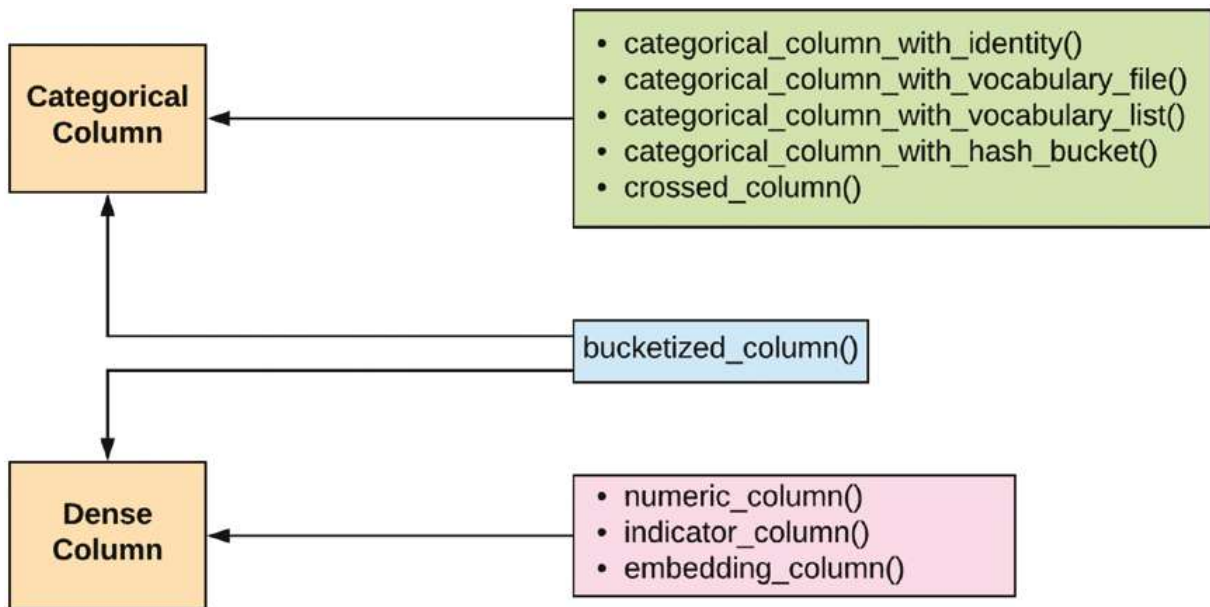


Figure 30-4. Function calls of the Feature Column API

Let's go through each API function briefly in Table 30-1.

Table 30-1. *tf.feature_column* API Functions

Function name	Description
Numeric column – tf.feature_column.numeric_column()	This is a high-level wrapper for numeric features in the dataset.
Indicator column – tf.feature_column.indicator_column()	The indicator column takes as input a categorical column and transforms it into a one-hot encoded vector.
Embedding column – tf.feature_column.embedding_column()	The embedding column function transforms a categorical column with multiple levels or classes into a lower-dimensional numeric representation that captures the relationships between the categories. Using embeddings mitigates the problem of a large sparse vector (an array with mostly zeros) created via one-hot encoding for a dataset feature with lots of different classes.

(continued)

Table 30-1. (continued)

Function name	Description
Categorical column with identity – tf.feature_column.categorical_column_with_identity()	This function creates a one-hot encoded output of a categorical column containing identities, e.g, ['0', '1', '2', '3'].
Categorical column with vocabulary list – tf.feature_column.categorical_column_with_vocabulary_list()	This function creates a one-hot encoded output of a categorical column with strings. It maps each string to an integer based on a vocabulary list. However, if the vocabulary list is long, it is best to create a file containing the vocabulary and use the function tf.feature_column.categorical_column_with_vocabulary_file() .
Categorical column with hash bucket – tf.feature_column.categorical_column_with_hash_buckets()	This function specifies the number of categories by using the hash of the inputs. It is used when it is not possible to create a vocabulary for the number of categories due to memory considerations.
Crossed column – tf.feature_columns.crossed_column()	The function gives the ability to combine multiple input features into a single input feature.
Bucketized column – tf.feature_column.bucketized_column()	The function splits a column of numerical inputs into buckets to form new classes based on a specified set of numerical ranges.

The High-Level TensorFlow APIs

The high-level API provides simplified API calls that encapsulate lots of the details that are typically involved in creating a deep learning TensorFlow model. These high-level abstractions make it easier to develop powerful deep learning models quickly with fewer lines of code.

Estimator API

The Estimator API is a high-level TensorFlow functionality that is aimed at reducing the complexity involved in building machine learning models by exposing methods that abstract common models and processes. There are two ways of working with Estimators, and they include

- **Using the premade Estimators:** The premade Estimators are black box models made available by the TensorFlow team for building common machine learning/deep learning architectures such as linear regression/classification, Random forest regression/classification and deep neural networks for regression and classification. An illustration of the premade Estimators as subclasses of the Estimator class is shown in Figure 30-5.

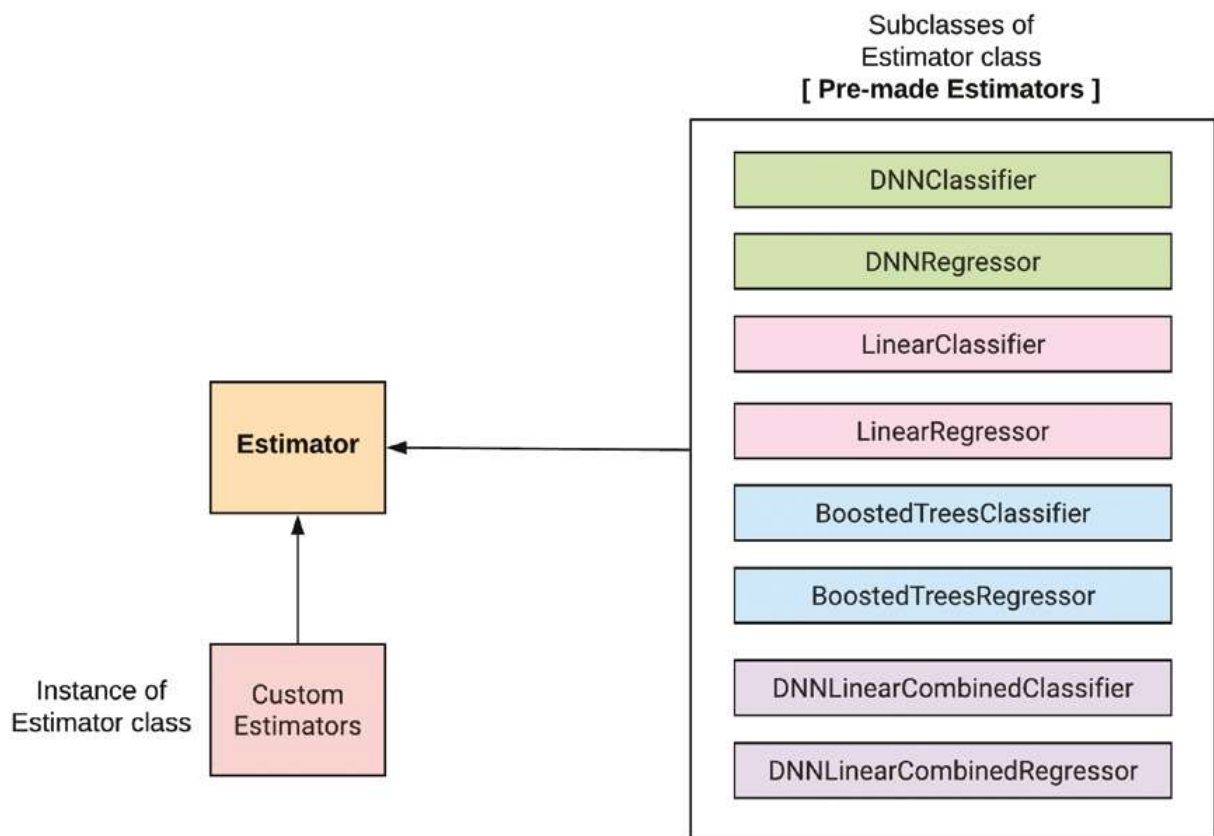


Figure 30-5. Estimator class API hierarchy

- Creating a custom Estimator:** It is also possible to use the low-level TensorFlow methods to create a custom black box model for easy reusability. To do this, you must put your code in a method called the **model_fn**. The model function will include code that defines operations such as the labels or predictions, loss function, the training operations, and the operations for evaluation.

The Estimator class exposes four major methods, namely, the **fit()**, **evaluate()**, **predict()**, and **export_savedmodel()** methods. The **fit()** method is called to train the data by running a loop of training operations. The **evaluate()** method is called to evaluate the model performance by looping through a set of evaluation operations. The **predict()** method uses the trained model to make predictions, while the **export_savedmodel()** method is used for exporting the trained model to a specified directory. For both the premade and custom Estimators, we must write a method to build the data input pipeline into the model. This pipeline is built for both the training and evaluation data inputs. This is further illustrated in Figure 30-6.

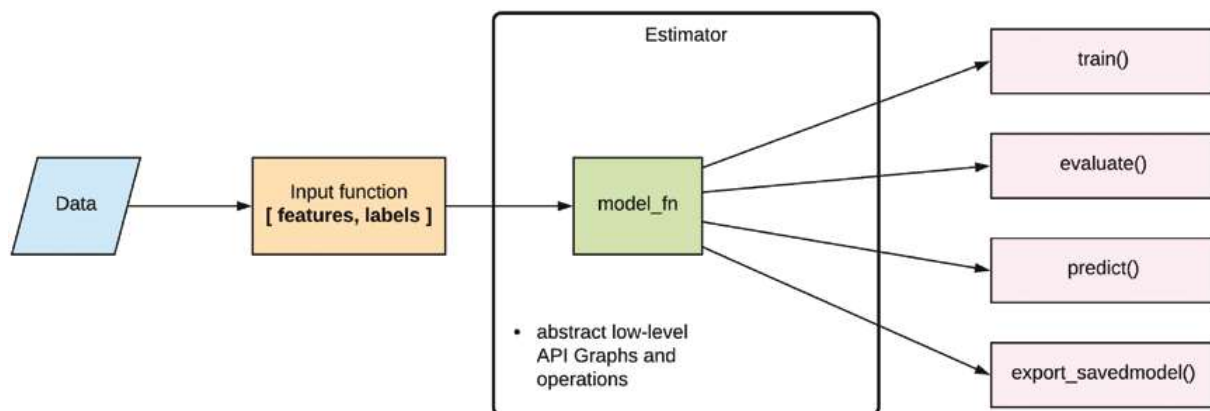


Figure 30-6. Estimator data input pipeline

Keras API

Keras provides a high-level specification for developing deep neural network models. The Keras API was initially separate from TensorFlow and only provided an interface for model building with TensorFlow as one of the frameworks running at the backend. However, in TensorFlow 2.0, Keras is an integral part of the TensorFlow codebase as preferred high-level API.

The Keras API version internal to TensorFlow is available from the 'tf.keras' package, whereas the broader Keras API blueprint that is not tied to a specific backend will remain available from the 'keras' package. In summary, when working with the 'keras' package, the backend can run with either TensorFlow, Microsoft CNTK, or Theano. On the other hand, working with 'tf.keras' provides a TensorFlow only version which is tightly integrated and compatible with all of the functionality of the core TensorFlow library.

In this book, we will focus on '**tf.Keras**' as a high-level API of TensorFlow.

The Anatomy of a Keras Program

The Keras '**Model**' forms the core of a Keras program. A 'Model' is first constructed, then it is compiled. Next, the compiled model is trained and evaluated using their respective training and evaluation datasets. Upon successful evaluation using the relevant metrics, the model is then used for making predictions on previously unseen data samples.

Figure 30-7 shows the program flow for modeling with Keras.

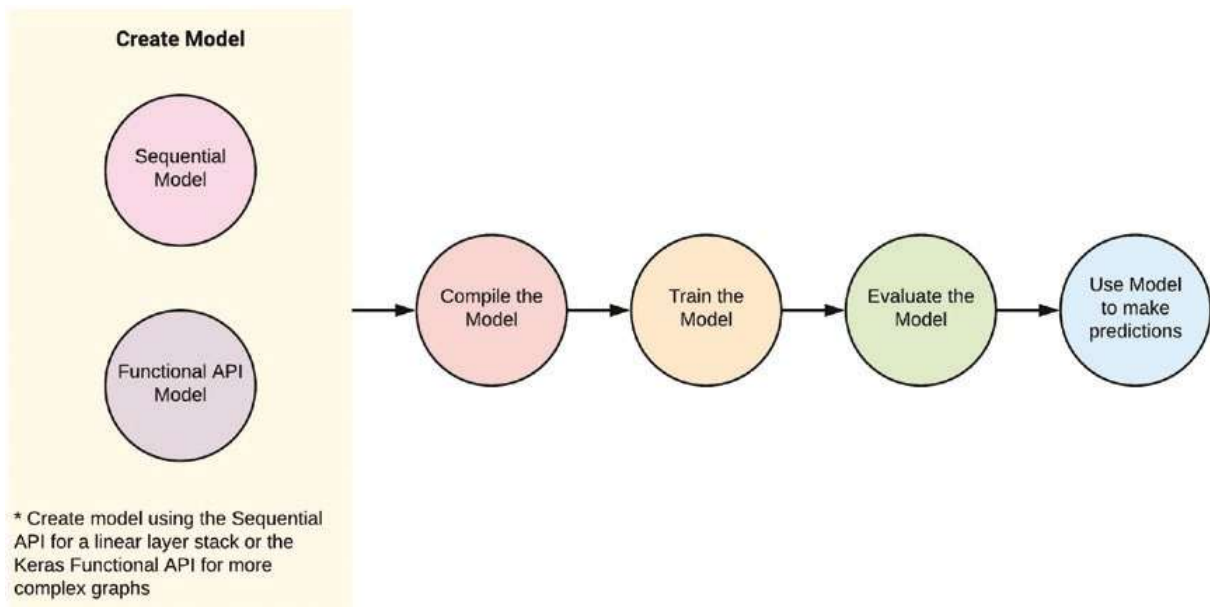


Figure 30-7. *The anatomy of a Keras program*

As shown in Figure 30-7, the Keras 'Model' can be constructed using the Sequential API 'tf.keras.Sequential' or the Keras Functional API which defines a model instance 'tf.keras.Model'. The Sequential model is the simplest method for creating a linear stack of

neural network layers. The Functional model is used if a more complex graph is desired. Keras is the de facto API for building neural network architectures with TensorFlow.

From here on, the code examples in this book will use the Sequential API, Functional API, and Model subclassing methods for building neural network architectures with Keras. In doing this, the reader can play around with the various examples as samples to get a feel of how they work.

TensorBoard

TensorBoard is an interactive visualization tool that comes bundled with TensorFlow. The goal of TensorBoard is to gain a visual insight into how the computational graph is constructed and executed. This information provides greater visibility for understanding, optimizing, and debugging deep learning models.

TensorBoard has a variety of visualization dashboard, such as

- **Scalar dashboard:** This dashboard captures metrics that change with time, such as the loss of a model or other model evaluation metrics such as accuracy, precision, recall, f1, and so on.
- **Histogram dashboard:** This dashboard shows the histogram distribution for a Tensor as it has changed over time.
- **Distribution dashboard:** This dashboard is similar to the histogram dashboard. However, it displays the histogram as a distribution.
- **Graph explorer:** This dashboard gives a graphical overview of the TensorFlow computational graph and how information flows from one node to the other. This dashboard provides invaluable insights into the network architecture.
- **Image dashboard:** This dashboard displays images saved using the method **`tf.summary.image`**.
- **Audio dashboard:** This dashboard provides audio clips saved using the method **`tf.summary.audio`**.

- **Embedding projector:** The dashboard makes it easy to visualize high-dimensional datasets after they have been transformed using **Embeddings**. The visualization uses principal component analysis (PCA) and another technique called t-distributed Stochastic Neighbor Embedding (t-SNE). Embedding is a technique for capturing the latent variables in a high-dimensional dataset by converting the data units into real numbers that capture their relationship. This technique is broadly similar to how PCA reduces data dimensionality. Embeddings are also useful for converting sparse matrices (matrices made up of mostly zeros) into a dense representation.
- **Text dashboard:** This dashboard is for displaying textual information.

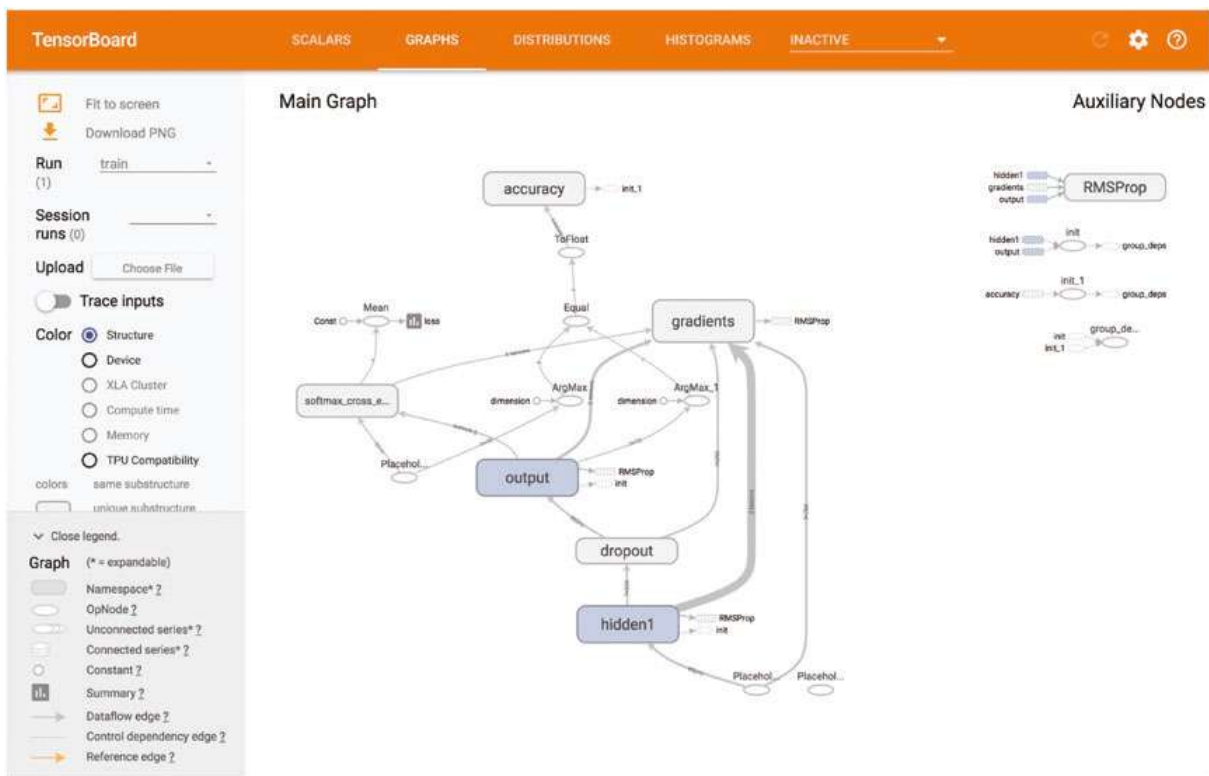


Figure 30-8. TensorBoard

Features in TensorFlow 2.0

TensorFlow 2.0 comes with new features for building machine learning models. Some of these new features include

- A more pythonic feel to model design and debugging with eager execution as the de facto execution mode.
- Eager execution enables instant evaluation of TensorFlow operations. This is opposed to previous versions of Tensorflow where we first construct a computational graph and then execute it in a session.
- Using `tf.function` to transform a Python method into high-performance TensorFlow graphs.
- Using Keras as the core high-level API for model design.
- Using FeatureColumns to parse data as input into Keras models.
- The ease of training on distributed architectures and devices.

To install and work with TensorFlow 2.0 on Google Colab, run

```
!pip install -q tensorflow==2.0.0-beta0
```

The GCP Deep Learning VM has images with TensorFlow 2.0 pre-configured.

A Simple TensorFlow Program

Let's start by building a simple TF program. Here, we will build a graph to find the roots of the quadratic expression $x^2 + 3x - 4 = 0$.

```
# import tensorflow
import tensorflow as tf

# Quadratic expression: x**2 + 3x - 4 = 0.
a = tf.constant(1.0)
b = tf.constant(3.0)
c = tf.constant(-4.0)
```

```

print(a)
print(b)
print(c)

'Output':
tf.Tensor(1.0, shape=(), dtype=float32)
tf.Tensor(3.0, shape=(), dtype=float32)
tf.Tensor(-4.0, shape=(), dtype=float32)

```

tf.constant() is a Tensor for storing a constant type. Now let's calculate the roots of the expression.

```

x1 = (-b + tf.math.sqrt(b**2 - (4*a*c))) / 2**a
x2 = (-b - tf.math.sqrt(b**2 - (4*a*c))) / 2**a

roots = (x1, x2)
print(roots)

```

```

'Output':
(<tf.Tensor: id=163, shape=(), dtype=float32, numpy=1.0>, <tf.Tensor:
id=175, shape=(), dtype=float32, numpy=-4.0>)

```

TensorFlow 2.0 is eager-first; this implies that operations are executed immediately after they are defined, just like regular python code.

Building Efficient Input Pipelines with the Dataset API

The Dataset API '**tf.data**' offers an efficient mechanism for building robust input pipelines for passing data into a TensorFlow program. This section uses the Boston housing dataset to illustrate working with the Dataset API methods for building data input pipelines in TensorFlow.

```

# import packages
import tensorflow as tf
from tensorflow.keras.datasets import boston_housing

# load dataset and split in train and test sets
(X_train, y_train), (X_test, y_test) = boston_housing.load_data()

```

```

# construct data input pipelines
dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train))
dataset = dataset.shuffle(buffer_size=1000)
dataset = dataset.batch(5)

# retrieve first data batch from dataset
for features, labels in dataset:
    print('Features:', features)
    print('Shape of Features:', features.shape)

    print('Labels:', labels)
    print('Shape of Labels:', labels.shape)
    break

'Output':
Features: tf.Tensor(
[[8.19900e-02 0.00000e+00 1.39200e+01 0.00000e+00 4.37000e-01 6.00900e+00
 4.23000e+01 5.50270e+00 4.00000e+00 2.89000e+02 1.60000e+01 3.96900e+02
 1.04000e+01]
 [8.82900e-02 1.25000e+01 7.87000e+00 0.00000e+00 5.24000e-01 6.01200e+00
 6.66000e+01 5.56050e+00 5.00000e+00 3.11000e+02 1.52000e+01 3.95600e+02
 1.24300e+01]
 [2.90900e-01 0.00000e+00 2.18900e+01 0.00000e+00 6.24000e-01 6.17400e+00
 9.36000e+01 1.61190e+00 4.00000e+00 4.37000e+02 2.12000e+01 3.88080e+02
 2.41600e+01]
 [5.87205e+00 0.00000e+00 1.81000e+01 0.00000e+00 6.93000e-01 6.40500e+00
 9.60000e+01 1.67680e+00 2.40000e+01 6.66000e+02 2.02000e+01 3.96900e+02
 1.93700e+01]
 [1.71710e-01 2.50000e+01 5.13000e+00 0.00000e+00 4.53000e-01 5.96600e+00
 9.34000e+01 6.81850e+00 8.00000e+00 2.84000e+02 1.97000e+01 3.78080e+02
 1.44400e+01]], shape=(5, 13), dtype=float64)
Shape of Features: (5, 13)
Labels: tf.Tensor([21.7 22.9 14.  12.5 16. ], shape=(5,)), dtype=float64)
Shape of Labels: (5,)

```

From the preceding code listing, take note of the following:

- The method **'tf.data.Dataset.from_tensor_slices()'** is used to create a Dataset whose elements are Tensor slices.
- The Dataset method **'shuffle()'** shuffles the Dataset at each epoch.
- The Dataset method **'batch()'** is used to set the size of each mini-batch of the Dataset. In the preceding example, each Dataset batch contains five observations.

Linear Regression with TensorFlow

In this section, we use TensorFlow to implement a linear regression machine learning model. In the following example, we use the Boston house-prices dataset from the **Keras dataset package** to build a linear regression model with TensorFlow 2.0.

```
# import packages
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import boston_housing
from tensorflow.keras import Model
from sklearn.preprocessing import StandardScaler

# load dataset and split in train and test sets
(X_train, y_train), (X_test, y_test) = boston_housing.load_data()

# standardize the dataset
scaler_X_train = StandardScaler().fit(X_train)
scaler_X_test = StandardScaler().fit(X_test)
X_train = scaler_X_train.transform(X_train)
X_test = scaler_X_test.transform(X_test)

# reshape y-data to become column vector
y_train = np.reshape(y_train, [-1, 1])
y_test = np.reshape(y_test, [-1, 1])

# build the linear model
class LinearRegressionModel(Model):
```



```

def __init__(self):
    super(LinearRegressionModel, self).__init__()
    # initialize weight and bias variables
    self.weight = tf.Variable(
        initial_value = tf.random.normal(
            [13, 1], dtype=tf.float64),
        trainable=True)
    self.bias = tf.Variable(initial_value = tf.constant(
        1.0, shape=[], dtype=tf.float64), trainable=True)

def call(self, inputs):
    return tf.add(tf.matmul(inputs, self.weight), self.bias)

model = LinearRegressionModel()

# parameters
batch_size = 32
learning_rate = 0.01

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (X_train, y_train)).shuffle(len(X_train)).batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(batch_size)

loss_object = tf.keras.losses.MeanSquaredError()

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)

train_loss = tf.keras.metrics.Mean(name='train_loss')
train_rmse = tf.keras.metrics.RootMeanSquaredError(name='train_rmse')

test_loss = tf.keras.metrics.Mean(name='test_loss')
test_rmse = tf.keras.metrics.RootMeanSquaredError(name='test_rmse')

# use tf.GradientTape to train the model
@tf.function
def train_step(inputs, labels):
    with tf.GradientTape() as tape:
        predictions = model(inputs)
        loss = loss_object(labels, predictions)

```

```

gradients = tape.gradient(loss, model.trainable_variables)
optimizer.apply_gradients(zip(gradients, model.trainable_variables))

train_loss(loss)
train_rmse(labels, predictions)

@tf.function
def test_step(inputs, labels):
    predictions = model(inputs)
    t_loss = loss_object(labels, predictions)

    test_loss(t_loss)
    test_rmse(labels, predictions)

num_epochs = 1000

for epoch in range(num_epochs):
    for train_inputs, train_labels in train_ds:
        train_step(train_inputs, train_labels)

    for test_inputs, test_labels in test_ds:
        test_step(test_inputs, test_labels)

    template = 'Epoch {}, Loss: {}, RMSE: {}, Test Loss: {}, Test RMSE: {}'

    if ((epoch+1) % 100 == 0):
        print (template.format(epoch+1,
                                train_loss.result(),
                                train_rmse.result(),
                                test_loss.result(),
                                test_rmse.result()))

'Output':
Epoch 100, Loss: 23.531124114990234, RMSE: 4.862841606140137, Test Loss:
21.077274322509766, Test RMSE: 4.591667175292969
Epoch 200, Loss: 23.51316261291504, RMSE: 4.860987663269043, Test Loss:
21.067768096923828, Test RMSE: 4.590633869171143
Epoch 300, Loss: 23.496540069580078, RMSE: 4.859271049499512, Test Loss:
21.058971405029297, Test RMSE: 4.589677333831787

```

Epoch 400, Loss: 23.481115341186523, RMSE: 4.857677459716797, Test Loss: 21.050806045532227, Test RMSE: 4.588788986206055
 Epoch 500, Loss: 23.466760635375977, RMSE: 4.856194019317627, Test Loss: 21.043209075927734, Test RMSE: 4.587962627410889
 Epoch 600, Loss: 23.453369140625, RMSE: 4.8548102378845215, Test Loss: 21.036123275756836, Test RMSE: 4.587191581726074
 Epoch 700, Loss: 23.440847396850586, RMSE: 4.853515625, Test Loss: 21.029495239257812, Test RMSE: 4.586470603942871
 Epoch 800, Loss: 23.429113388061523, RMSE: 4.852302074432373, Test Loss: 21.02336311340332, Test RMSE: 4.585799694061279
 Epoch 900, Loss: 23.4180965423584, RMSE: 4.851161956787109, Test Loss: 21.017648696899414, Test RMSE: 4.585177898406982
 Epoch 1000, Loss: 23.407730102539062, RMSE: 4.8500895500183105, Test Loss: 21.012271881103516, Test RMSE: 4.584592819213867

Here are a few points and methods to take note of in the preceding code listing for linear regression with TensorFlow:

- Note that transformation to standardize the feature dataset is performed after splitting the data into train and test sets. This action is performed in this manner to prevent information from the training data to pollute the test data which must remain unseen by the model.
- The class named **'LinearRegressionModel'** builds a Keras model by subclassing the **'tf.keras.Model'** class. The linear regression model is created as a layer of the neural network in the **'__init__'** method, and it is defined as a forward pass in the **'call'** method. In Chapter 31 on Keras, we will see how to use simpler routines with the Keras Functional API.
- The **'tf.data.Dataset.from_tensor_slices'** method uses the **'minimize()'** method to update the loss function.
- The squared error loss function is defined with **'tf.keras.losses.MeanSquaredError()'**.
- The gradient descent optimization algorithm is defined using **'tf.keras.optimizers.SGD()'** with the learning rate set as a parameter to the method.

- The method to capture the loss and root mean squared error estimates is defined using `'tf.keras.metrics.Mean(name='train_loss')'` and `'tf.keras.metrics.RootMeanSquaredError()'` functions, respectively.
- The `@tf.function` is a python decorator to transform a method into high-performance TensorFlow graphs.
- The method `'train_step'` uses the `'tf.GradientTape()'` method to record operations for automatic differentiation. These gradients are later used to minimize the cost function by calling the `'apply_gradients()'` method of the optimization algorithm.
- The method `'test_step'` uses the trained model to obtain predictions on test data.

Classification with TensorFlow

In this example, we'll use the Iris flower dataset to build a multivariable logistic regression machine learning classifier with TensorFlow 2.0. The dataset is gotten from the Scikit-learn dataset package.

```
# import packages
import numpy as np
import tensorflow as tf
from sklearn import datasets
from tensorflow.keras import Model
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# apply one-hot encoding to targets
one_hot_encoder = OneHotEncoder(categories='auto')
```

```

encode_categorical = y.reshape(len(y), 1)
y = one_hot_encoder.fit_transform(encode_categorical).toarray()

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# build the linear model
class LogisticRegressionModel(Model):
    def __init__(self):
        super(LogisticRegressionModel, self).__init__()
        # initialize weight and bias variables
        self.weight = tf.Variable(
            initial_value = tf.random.normal(
                [4, 3], dtype=tf.float64),
            trainable=True)
        self.bias = tf.Variable(initial_value = tf.random.normal(
            [3], dtype=tf.float64), trainable=True)

    def call(self, inputs):
        return tf.add(tf.matmul(inputs, self.weight), self.bias)

model = LogisticRegressionModel()

# parameters
batch_size = 32
learning_rate = 0.1

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (X_train, y_train)).shuffle(len(X_train)).batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(batch_size)

optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)

train_loss = tf.keras.metrics.Mean(name='train_loss')
train_accuracy = tf.keras.metrics.Accuracy(name='train_accuracy')

test_loss = tf.keras.metrics.Mean(name='test_loss')
test_accuracy = tf.keras.metrics.Accuracy(name='test_accuracy')

```

```

# use tf.GradientTape to train the model
@tf.function
def train_step(inputs, labels):
    with tf.GradientTape() as tape:
        predictions = model(inputs)
        loss = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(labels,
            predictions))
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

    train_loss(loss)
    train_accuracy(tf.argmax(labels,1), tf.argmax(predictions,1))

@tf.function
def test_step(inputs, labels):
    predictions = model(inputs)
    t_loss = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(labels,
        predictions))

    test_loss(t_loss)
    test_accuracy(tf.argmax(labels,1), tf.argmax(predictions,1))

num_epochs = 1000

for epoch in range(num_epochs):
    for train_inputs, train_labels in train_ds:
        train_step(train_inputs, train_labels)

    for test_inputs, test_labels in test_ds:
        test_step(test_inputs, test_labels)

    template = 'Epoch {}, Loss: {}, Accuracy: {}, Test Loss: {}, Test Accuracy: {}'

    if ((epoch+1) % 100 == 0):
        print (template.format(epoch+1,
                                train_loss.result(),
                                train_accuracy.result()*100,
                                test_loss.result(),
                                test_accuracy.result()*100))

```

'Output':

```
Epoch 100, Loss: 0.3510790765285492, Accuracy: 89.63029479980469, Test
Loss: 0.44924452900886536, Test Accuracy: 84.37885284423828
Epoch 200, Loss: 0.3282322287559509, Accuracy: 91.29582214355469, Test
Loss: 0.43276602029800415, Test Accuracy: 85.73675537109375
Epoch 300, Loss: 0.3093726634979248, Accuracy: 92.46343231201172, Test
Loss: 0.41915151476860046, Test Accuracy: 86.6886978149414
Epoch 400, Loss: 0.29340484738349915, Accuracy: 93.3273696899414, Test
Loss: 0.40762627124786377, Test Accuracy: 87.43070220947266
Epoch 500, Loss: 0.2796294391155243, Accuracy: 93.99247741699219, Test
Loss: 0.3976936936378479, Test Accuracy: 88.27145385742188
Epoch 600, Loss: 0.2675718069076538, Accuracy: 94.52030944824219, Test
Loss: 0.38901543617248535, Test Accuracy: 88.93867492675781
Epoch 700, Loss: 0.25689396262168884, Accuracy: 94.94937896728516, Test
Loss: 0.38134896755218506, Test Accuracy: 89.48106384277344
Epoch 800, Loss: 0.24734711647033691, Accuracy: 95.3050537109375, Test
Loss: 0.3745149075984955, Test Accuracy: 89.9306640625
Epoch 900, Loss: 0.23874221742153168, Accuracy: 95.60466766357422, Test
Loss: 0.3683767020702362, Test Accuracy: 90.30940246582031
Epoch 1000, Loss: 0.23093272745609283, Accuracy: 95.86051177978516, Test
Loss: 0.3628271818161011, Test Accuracy: 90.63280487060547
```

From the preceding code, listing is similar to the example on linear regression with TensorFlow 2.0. However, take note of the following procedures:

- The target variable 'y' is converted to a one-hot encoded matrix by using the **'OneHotEncoder'** function from Scikit-learn. There exists a TensorFlow method named **'tf.one_hot'** for performing the same function, even easier! The reader is encouraged to Experiment with this.
- Observe how the **'tf.reduce_mean'** and the **'tf.nn.softmax_cross_entropy_with_logits'** methods are used to implement the loss for optimizing the logistic model.
- The Stochastic Gradient Descent optimization algorithm **'tf.keras.optimizers.SGD()'** is used to train the logistic model.

- Observe how the **‘weight’** and **‘bias’** variables are updated by the gradient descent optimizer within the **‘train_step’** method using **‘tf.GradientTape()’** to capture and compute the derivatives from the trainable model variables.
- The **‘tf.keras.metrics.Accuracy’** method is used to evaluate the accuracy of the model.

Visualizing with TensorBoard

In this section, we will go through visualizing TensorFlow graphs and statistics with TensorBoard. The following code improves on the previous code to build a linear regression model by adding methods to visualize the graph and other variable statistics in TensorBoard using the **‘tf.summary’** method calls. The TensorBoard output (illustrated in Figure 30-9) is displayed within the notebook.

```
# import packages
import datetime
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import boston_housing
from tensorflow.keras import Model
from sklearn.preprocessing import StandardScaler

# load the TensorBoard notebook extension
%load_ext tensorboard

# load dataset and split in train and test sets
(X_train, y_train), (X_test, y_test) = boston_housing.load_data()

# standardize the dataset
scaler_X_train = StandardScaler().fit(X_train)
scaler_X_test = StandardScaler().fit(X_test)
X_train = scaler_X_train.transform(X_train)
X_test = scaler_X_test.transform(X_test)

# reshape y-data to become column vector
y_train = np.reshape(y_train, [-1, 1])
```



```

y_test = np.reshape(y_test, [-1, 1])

# build the linear model
class LinearRegressionModel(Model):
    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        # initialize weight and bias variables
        self.weight = tf.Variable(
            initial_value = tf.random.normal(
                [13, 1], dtype=tf.float64),
            trainable=True)
        self.bias = tf.Variable(initial_value = tf.constant(
            1.0, shape=[], dtype=tf.float64), trainable=True)

    def call(self, inputs):
        return tf.add(tf.matmul(inputs, self.weight), self.bias)

model = LinearRegressionModel()

# parameters
batch_size = 32
learning_rate = 0.01

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (X_train, y_train)).shuffle(len(X_train)).batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(batch_size)

loss_object = tf.keras.losses.MeanSquaredError()
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)

train_loss = tf.keras.metrics.Mean(name='train_loss')
train_rmse = tf.keras.metrics.RootMeanSquaredError(name='train_rmse')

test_loss = tf.keras.metrics.Mean(name='test_loss')
test_rmse = tf.keras.metrics.RootMeanSquaredError(name='test_rmse')

```

```

# use tf.GradientTape to train the model
@tf.function
def train_step(inputs, labels):
    with tf.GradientTape() as tape:
        predictions = model(inputs)
        loss = loss_object(labels, predictions)
        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))

    train_loss(loss)
    train_rmse(labels, predictions)

@tf.function
def test_step(inputs, labels):
    predictions = model(inputs)
    t_loss = loss_object(labels, predictions)

    test_loss(t_loss)
    test_rmse(labels, predictions)

# Clear any logs from previous runs
!rm -rf ./logs/

# set up summary writers to write the summaries to disk in a different logs
directory
current_time = datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
train_log_dir = 'logs/gradient_tape/' + current_time + '/train'
test_log_dir = 'logs/gradient_tape/' + current_time + '/test'
train_summary_writer = tf.summary.create_file_writer(train_log_dir)
test_summary_writer = tf.summary.create_file_writer(test_log_dir)

num_epochs = 1000

for epoch in range(num_epochs):
    for train_inputs, train_labels in train_ds:
        train_step(train_inputs, train_labels)
    with train_summary_writer.as_default():
        tf.summary.scalar('loss', train_loss.result(), step=epoch)
        tf.summary.scalar('rmse', train_rmse.result(), step=epoch)

```

```

for test_inputs, test_labels in test_ds:
    test_step(test_inputs, test_labels)
with test_summary_writer.as_default():
    tf.summary.scalar('loss', test_loss.result(), step=epoch)
    tf.summary.scalar('rmse', test_rmse.result(), step=epoch)

template = 'Epoch {}, Loss: {}, RMSE: {}, Test Loss: {}, Test RMSE: {}'

if ((epoch+1) % 100 == 0):
    print (template.format(epoch+1,
                           train_loss.result(),
                           train_rmse.result(),
                           test_loss.result(),
                           test_rmse.result()))

# Reset metrics every epoch
train_loss.reset_states()
test_loss.reset_states()
train_rmse.reset_states()
test_rmse.reset_states()

'Output':
Epoch 100, Loss: 22.03757667541504, RMSE: 4.726028919219971, Test Loss:
29.092111587524414, Test RMSE: 4.577760696411133
Epoch 200, Loss: 21.973844528198242, RMSE: 4.719051837921143, Test Loss:
29.113895416259766, Test RMSE: 4.585252285003662
Epoch 300, Loss: 21.970674514770508, RMSE: 4.7187066078186035, Test Loss:
29.13644790649414, Test RMSE: 4.587917327880859
Epoch 400, Loss: 21.970500946044922, RMSE: 4.718687534332275, Test Loss:
29.1422119140625, Test RMSE: 4.588583469390869
Epoch 500, Loss: 21.970489501953125, RMSE: 4.718685626983643, Test Loss:
29.14352035522461, Test RMSE: 4.588735103607178
Epoch 600, Loss: 21.970487594604492, RMSE: 4.718685626983643, Test Loss:
29.143817901611328, Test RMSE: 4.58876895904541
Epoch 700, Loss: 21.970487594604492, RMSE: 4.718685626983643, Test Loss:
29.143882751464844, Test RMSE: 4.588776111602783

```

Epoch 800, Loss: 21.970487594604492, RMSE: 4.718685626983643, Test Loss:
29.14389419555664, Test RMSE: 4.588778018951416

Epoch 900, Loss: 21.970487594604492, RMSE: 4.718685626983643, Test Loss:
29.143898010253906, Test RMSE: 4.588778495788574

Epoch 1000, Loss: 21.970487594604492, RMSE: 4.718685626983643, Test Loss:
29.143898010253906, Test RMSE: 4.588778495788574

```
# launch tensorboard
```

```
%tensorboard --logdir logs/gradient_tape
```

From the preceding code listing, take note of the following steps:

- The **'tf.summary.create_file_writer'** method creates summary writers to write the summaries to disk.
- The **'tf.summary.scalar'** method is used to capture scalar metrics for TensorBoard.
- The magic command **'%tensorboard'** is used to launch TensorBoard by pointing to the appropriate log directory.

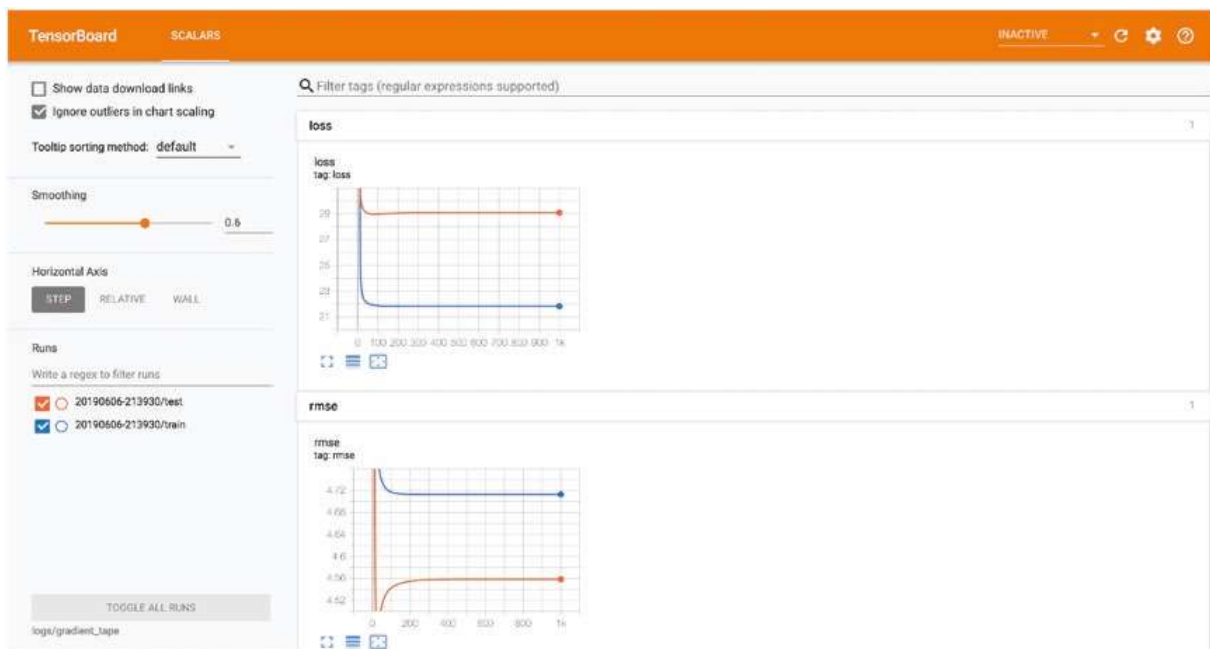


Figure 30-9. TensorBoard visualization dashboard for linear regression metrics

Running TensorFlow with GPUs

GPU is short for graphics processing unit. It is a specialized processor designed for carrying out complex computations on large memory blocks. GPUs provide more efficient processing for building deep learning models.

TensorFlow can leverage processing on multiple GPUs to speed up computation especially when training a complex network architecture. To take advantage of parallel processing, a replica of the network architecture resides on each GPU machine and trains a subset of the data. However, for synchronous updates, the model parameters from each tower (or GPU machines) are stored and updated on a CPU. It turns out that CPUs are generally good at mean or averaging processing. A diagram of this operation is shown in [Figure 30-10](#).

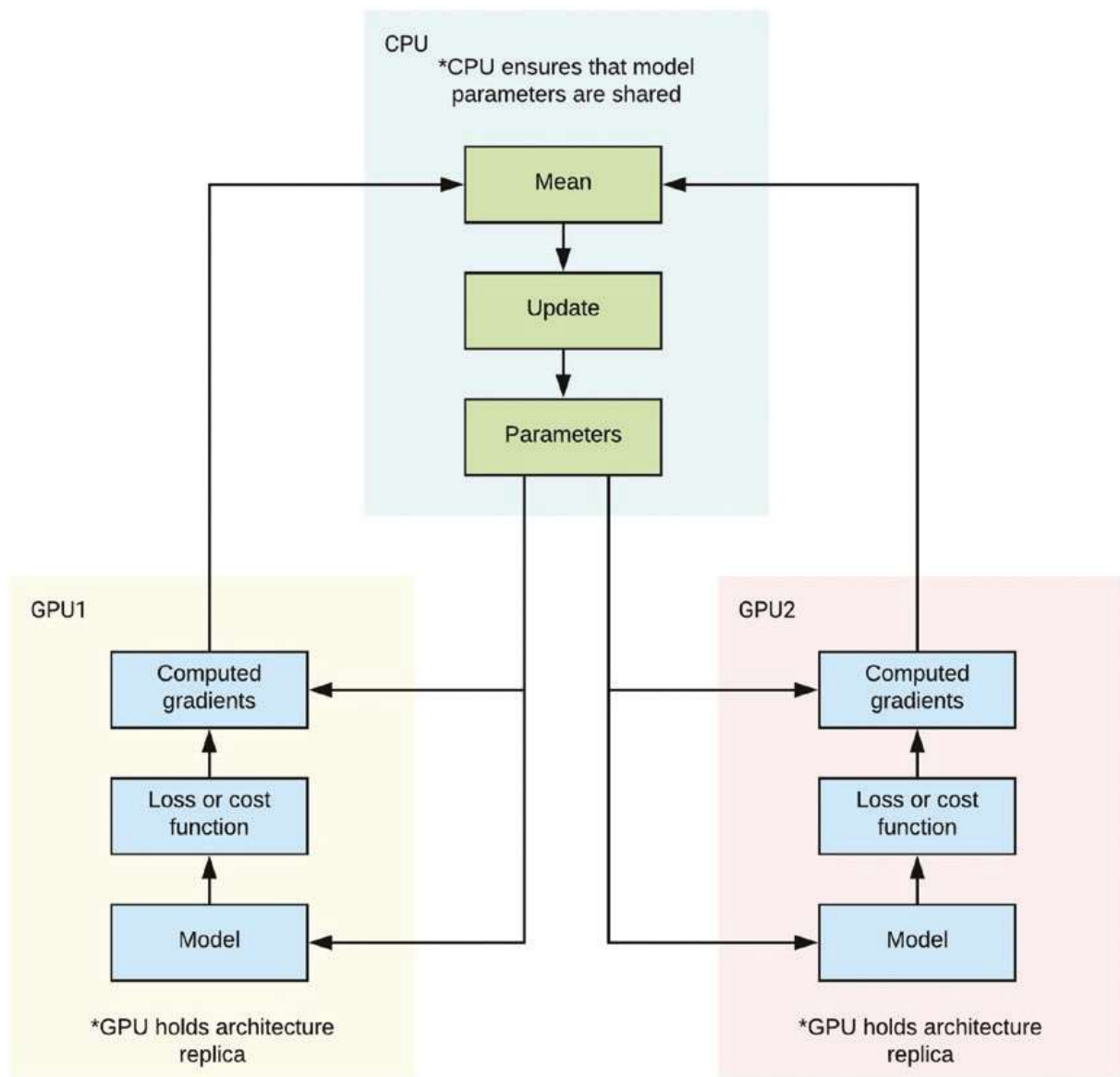


Figure 30-10. Framework for training on multiple GPUs

TensorFlow 2.0 performs distributed training across multiple machines (i.e., CPUs, GPUs, or TPUs) using the **'tf.distribute.Strategy'** API. To use GPUs on Google Colab, first change the runtime type to GPU and install the TensorFlow with GPU library by running the following code in the notebook cell:

```
!pip install -q tf-nightly-gpu-2.0-preview
```

The following code block uses GPUs for model training. In this example we train a simple regression model on the Boston housing dataset. The method **'tf.distribute.MirroredStrategy()'** implements a distribution strategy called **MirroredStrategy**. This strategy supports distributed training with multiple GPUs on a single machine. The code is similar to the previous code for linear regression with TensorFlow 2.0. However, minimal changes are added to make the components such as variables, layers, models, optimizers, metrics, summaries, and checkpoints strategy-aware using the `strategy.scope()`.

```
# import TensorFlow 2.0 with GPU
!pip install -q tf-nightly-gpu-2.0-preview

# confirm tensorflow can see GPU
import tensorflow as tf

device_name = tf.test.gpu_device_name()
if device_name != '/device:GPU:0':
    raise SystemError('GPU device not found')
print('Found GPU at: {}'.format(device_name))

# import other packages
import numpy as np
from tensorflow.keras.datasets import boston_housing
from tensorflow.keras import Model
from sklearn.preprocessing import StandardScaler

# load dataset and split in train and test sets
(X_train, y_train), (X_test, y_test) = boston_housing.load_data()

# standardize the dataset
scaler_X_train = StandardScaler().fit(X_train)
scaler_X_test = StandardScaler().fit(X_test)
X_train = scaler_X_train.transform(X_train)
X_test = scaler_X_test.transform(X_test)

# reshape y-data to become column vector
y_train = np.reshape(y_train, [-1, 1])
y_test = np.reshape(y_test, [-1, 1])
```

```

# build the linear model
class LinearRegressionModel(Model):
    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        # initialize weight and bias variables
        self.weight = tf.Variable(
            initial_value = tf.random.normal(
                [13, 1], dtype=tf.float64),
            trainable=True)
        self.bias = tf.Variable(initial_value = tf.constant(
            1.0, shape=[], dtype=tf.float64), trainable=True)

    def call(self, inputs):
        return tf.add(tf.matmul(inputs, self.weight), self.bias)

# create a strategy to distribute the variables and the graph
strategy = tf.distribute.MirroredStrategy()

# print number of machines with GPUs
print ('Number of devices: {}'.format(strategy.num_replicas_in_sync))

# parameters
batch_size_per_replica = 32
global_batch_size = batch_size_per_replica * strategy.num_replicas_in_sync

learning_rate = 0.01

# create the distributed datasets inside a strategy.scope:
with strategy.scope():
    train_ds = tf.data.Dataset.from_tensor_slices(
        (X_train, y_train)).shuffle(len(X_train)).batch(global_batch_size)
    train_dist_ds = strategy.experimental_distribute_dataset(train_ds)

    test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).
        batch(global_batch_size)
    test_dist_ds = strategy.experimental_distribute_dataset(test_ds)

```



```

# define the loss function
with strategy.scope():
    # Set reduction to `none` so we can do the reduction afterwards and
    # divide by
    # global batch size.
    loss_object = tf.keras.losses.MeanSquaredError(
        reduction=tf.keras.losses.Reduction.NONE)

    def compute_loss(labels, predictions):
        per_example_loss = loss_object(labels, predictions)
        return tf.reduce_sum(per_example_loss) * (1. / global_batch_size)

# define metrics to track loss and rmse
with strategy.scope():
    test_loss = tf.keras.metrics.Mean(name='test_loss')

    train_rmse = tf.keras.metrics.RootMeanSquaredError(
        name='train_rmse')
    test_rmse = tf.keras.metrics.RootMeanSquaredError(
        name='test_rmse')

# model and optimizer must be created under `strategy.scope`.
with strategy.scope():
    model = LinearRegressionModel()
    optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)

with strategy.scope():
    def train_step(inputs, labels):
        with tf.GradientTape() as tape:
            predictions = model(inputs)
            loss = compute_loss(labels, predictions)

        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))

        train_rmse.update_state(labels, predictions)
        return loss

```

```

def test_step(inputs, labels):
    predictions = model(inputs)
    t_loss = loss_object(labels, predictions)

    test_loss.update_state(t_loss)
    test_rmse.update_state(labels, predictions)

num_epochs = 1000

with strategy.scope():
    # `experimental_run_v2` replicates the provided computation and runs it
    # with the distributed input.
    @tf.function
    def distributed_train_step(inputs, labels):
        per_replica_losses = strategy.experimental_run_v2(train_step,
                                                            args=(inputs, labels))
        return strategy.reduce(tf.distribute.ReduceOp.SUM, per_replica_losses,
                               axis=None)

    @tf.function
    def distributed_test_step(inputs, labels):
        return strategy.experimental_run_v2(test_step, args=(inputs, labels))

    for epoch in range(num_epochs):
        # Train loop
        total_loss = 0.0
        num_batches = 0
        for train_inputs, train_labels in train_dist_ds:
            total_loss += distributed_train_step(train_inputs, train_labels)
            num_batches += 1
        train_loss = total_loss / num_batches

        # Test loop
        for test_inputs, test_labels in test_dist_ds:
            distributed_test_step(test_inputs, test_labels)

        if (epoch+1) % 100 == 0:
            template = ("Epoch {}, Loss: {}, RMSE: {}, Test Loss: {}, "
                        "Test RMSE: {}")

```

```

print (template.format(epoch+1, train_loss,
                        train_rmse.result(), test_loss.result(),
                        test_rmse.result()))

test_loss.reset_states()
train_rmse.reset_states()
test_rmse.reset_states()

'Output:'
Epoch 100, Loss: 21.673020569627965, RMSE: 4.724063396453857, Test Loss:
20.915191650390625, Test RMSE: 4.573312759399414
Epoch 200, Loss: 21.594741116702117, RMSE: 4.715524196624756, Test Loss:
20.994861602783203, Test RMSE: 4.582014560699463
Epoch 300, Loss: 21.590902259189097, RMSE: 4.7151055335998535, Test Loss:
21.02731704711914, Test RMSE: 4.585555076599121
Epoch 400, Loss: 21.59074064145569, RMSE: 4.715087413787842, Test Loss:
21.03565216064453, Test RMSE: 4.5864644050598145
Epoch 500, Loss: 21.590740279510765, RMSE: 4.715087413787842, Test Loss:
21.037595748901367, Test RMSE: 4.586676120758057
Epoch 600, Loss: 21.590742194311133, RMSE: 4.715087890625, Test Loss:
21.03803825378418, Test RMSE: 4.586724281311035
Epoch 700, Loss: 21.59074262401866, RMSE: 4.715087890625, Test Loss:
21.03813934326172, Test RMSE: 4.586735248565674
Epoch 800, Loss: 21.59074272223048, RMSE: 4.715087413787842, Test Loss:
21.038162231445312, Test RMSE: 4.586737632751465
Epoch 900, Loss: 21.59074286927267, RMSE: 4.715087413787842, Test Loss:
21.03816795349121, Test RMSE: 4.586737632751465
Epoch 1000, Loss: 21.590742907190307, RMSE: 4.715087413787842, Test Loss:
21.03816795349121, Test RMSE: 4.586738109588623

```

Please note the following from the preceding code block:

- When writing a custom training loop, sum the per example losses and divide the sum by the global batch size. In the code `tf.reduce_sum(per_example_loss) * (1. / global_batch_size)`. This needs to be done because after calculation on each replica, the gradients are synced across the replicas by summing them. When using `tf.keras.losses` classes, the loss reduction needs to be explicitly specified to be one of `NONE` or `SUM`.

TensorFlow High-Level APIs: Using Estimators

In this section, we will use the high-level TensorFlow Estimator API for modeling with premade Estimators. Estimators provide another high-level API for building TensorFlow models for execution on CPUs, GPUs, or TPUs with minimal code modification.

The following steps are typically followed when working with premade Estimators:

1. Write the **'input_fn'** to handle the data pipeline.
2. Define the type of data attributes into the model using feature columns **'tf.feature_column'**.
3. Instantiate one of the premade Estimators by passing in the feature columns and other relevant attributes.
4. Use the **'train()'**, **'evaluate()'**, and **'predict()'** methods to train and evaluate the model on evaluation dataset and use the model to make prediction/inference.

Let's see a simple example of working with a TensorFlow premade Estimator again using the Boston housing dataset.

Note Reset session before running the following cells and change runtime type to None.

```
# import packages
import datetime
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.datasets import boston_housing
from tensorflow.keras import Model
from sklearn.preprocessing import StandardScaler

# load dataset and split in train and test sets
(X_train, y_train), (X_test, y_test) = boston_housing.load_data()
```

```

# standardize the dataset
scaler_X_train = StandardScaler().fit(X_train)
scaler_X_test = StandardScaler().fit(X_test)
X_train = scaler_X_train.transform(X_train)
X_test = scaler_X_test.transform(X_test)

# reshape y-data to become column vector
y_train = np.reshape(y_train, [-1, 1])
y_test = np.reshape(y_test, [-1, 1])

# parameters
batch_size = 32
learning_rate = 0.01

# create an input_fn
def input_fn(features, labels, batch_size=30, training=True):
    dataset = tf.data.Dataset.from_tensor_slices((features, labels))
    if training:
        dataset = dataset.shuffle(buffer_size=1000)
        dataset = dataset.repeat()
    return dataset.batch(batch_size)

# use feature columns to define the attributes to the model
feature_columns = []
columns_names = []
for i in range(X_train.shape[1]):
    feature_columns.append(tf.feature_column.numeric_column(key=str(i)))
    columns_names.append(str(i))

# instantiate a LinearRegressor Estimator
estimator = tf.estimator.DNNRegressor(
    feature_columns=feature_columns,
    hidden_units=[20]
)

# convert feature datasets to dictionary
X_train_pd = pd.DataFrame(X_train)
X_train_pd.columns = columns_names

```

```

X_test_pd = pd.DataFrame(X_test)
X_test_pd.columns = columns_names

# train model
estimator.train(input_fn=lambda:input_fn(dict(X_train_pd), y_train),
steps=2000)

# evaluate model
metrics = estimator.evaluate(input_fn=lambda:input_fn(dict(X_test_pd),
y_test, training=False))

# print model metrics
metrics

```

Neural Networks with Keras

In this section, we will use the Sequential and Functional Keras API to build a simple neural network model. A Sequential API is the most commonly used method to build deep neural network models by stacking one layer on another. The Functional API offers more flexibility to build more complex neural network architectures. Both API methods are relatively easy to construct in Keras as we will see in the examples.

Subclassing a model as we did in the preceding examples provides even more flexibility for building and inspecting complex models. However, the code is more verbose and may be prone to errors. This technique should be used when it makes the most sense to, depending on the problem use case. We used them previously to serve as an illustration.

The following examples will use the Iris Dataset to build a neural network with one hidden layer as illustrated in Figure [30-11](#).

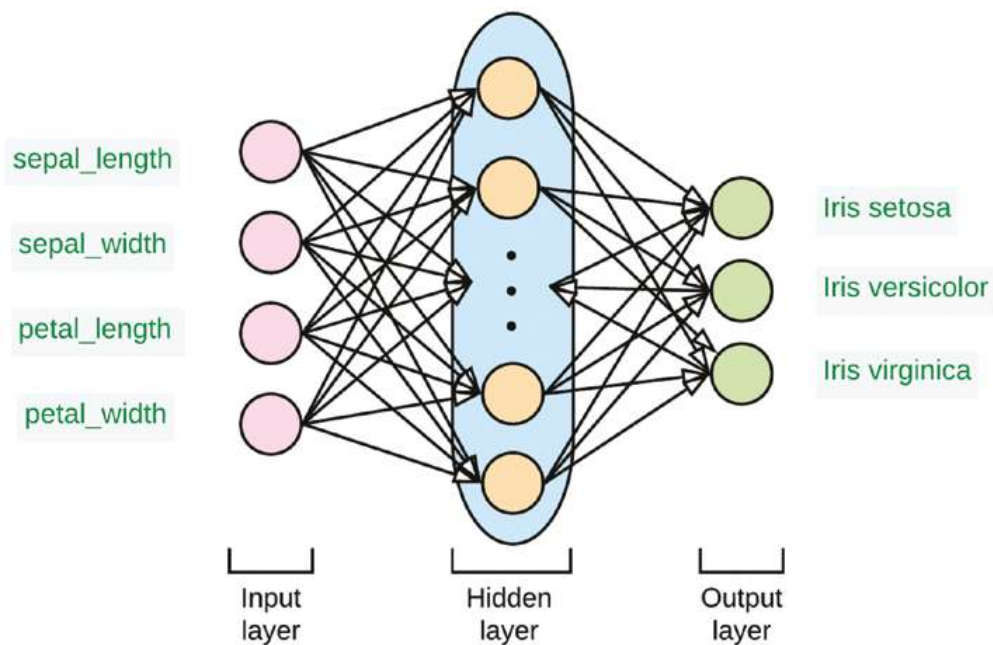


Figure 30-11. *Iris dataset – neural network architecture*

Using the Keras Sequential API

This code segment will construct a neural network model with the Sequential API using the method ‘`tf.keras.Sequential()`’ to stack layers on each other. The model creates a hidden layer with 32 neurons and an output layer with 3 output units because the Iris target contains 3 classes.

```
!pip install -q tensorflow==2.0.0-beta0

# import packages
import tensorflow as tf
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# dataset url
train_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_training.csv"
test_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_test.csv"
```

```

# define column names
columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']

# download and load the csv files
train_data = pd.read_csv(tf.keras.utils.get_file('iris_train.csv', train_
data_url),
                        skiprows=1, header=None, names=columns)

test_data = pd.read_csv(tf.keras.utils.get_file('iris_test.csv', test_data_url),
                        skiprows=1, header=None, names=columns)

# separate the features and targets
(X_train, y_train) = (train_data.iloc[:,0:-1], train_data.iloc[:,-1])
(X_test, y_test) = (test_data.iloc[:,0:-1], test_data.iloc[:,-1])

# apply one-hot encoding to targets
y_train=tf.keras.utils.to_categorical(y_train)
y_test=tf.keras.utils.to_categorical(y_test)

# create the sequential model
def model_fn():
    model = tf.keras.Sequential()
    # Add a densely-connected layer with 32 units to the model:
    model.add(tf.keras.layers.Dense(32, activation='sigmoid', input_dim=4))
    # Add a softmax layer with 3 output units:
    model.add(tf.keras.layers.Dense(3, activation='softmax'))

    # compile the model
    model.compile(optimizer=tf.keras.optimizers.SGD(),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model

# parameters
batch_size=50

```



```

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (X_train.values, y_train)).shuffle(len(X_train)).repeat().batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((X_test.values, y_test)).
    batch(batch_size)

# build train model
model = model_fn()

# print train model summary
model.summary()

# train the model
history = model.fit(train_ds, steps_per_epoch=5000)

# evaluate the model
score = model.evaluate(test_ds)

print('Test loss: {:.2f} \nTest accuracy: {:.2f}%'.format(score[0],
    score[1]*100))

'Output':
Test loss: 0.22
Test accuracy: 96.67%

```

Using the Keras Functional API

The general code pattern for the Functional API is structurally the same as the Sequential version. The only change here is in how the network model is constructed. We also demonstrated the Keras feature for printing the graph of the model in this example. The output is illustrated in [Figure 30-12](#).

```

!pip install -q tensorflow==2.0.0-beta0

# import packages
import tensorflow as tf
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

```

```

# dataset url
train_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_training.csv"
test_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_test.csv"

# define column names
columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']

# download and load the csv files
train_data = pd.read_csv(tf.keras.utils.get_file('iris_train.csv',
train_data_url),
                        skiprows=1, header=None, names=columns)

test_data = pd.read_csv(tf.keras.utils.get_file('iris_test.csv', test_data_url),
                        skiprows=1, header=None, names=columns)

# separate the features and targets
(X_train, y_train) = (train_data.iloc[:,0:-1], train_data.iloc[:, -1])
(X_test, y_test) = (test_data.iloc[:,0:-1], test_data.iloc[:, -1])

# apply one-hot encoding to targets
y_train=tf.keras.utils.to_categorical(y_train)
y_test=tf.keras.utils.to_categorical(y_test)

# create the functional model
def model_fn():
    # Model input
    model_input = tf.keras.layers.Input(shape=(4,))
    # Adds a densely-connected layer with 32 units to the model:
    x = tf.keras.layers.Dense(32, activation='relu')(model_input)
    # Add a softmax layer with 3 output units:
    predictions = tf.keras.layers.Dense(3, activation='softmax')(x)

    # the model
    model = tf.keras.Model(inputs=model_input,
                           outputs=predictions,
                           name='iris_model')

```

```

    # compile the model
    model.compile(optimizer='sgd',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model

# parameters
batch_size=50

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (X_train.values, y_train)).shuffle(len(X_train)).repeat().batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((X_test.values, y_test)).
batch(batch_size)

# build train model
model = model_fn()

# print train model summary
model.summary()

# plot the model as a graph
tf.keras.utils.plot_model(model, 'keras_iris_model.png', show_shapes=True)

# train the model
history = model.fit(train_ds, steps_per_epoch=5000)

# evaluate the model
score = model.evaluate(test_ds)

print('Test loss: {:.2f} \nTest accuracy: {:.2f}%'.format(score[0],
score[1]*100))

'Output':
Test loss: 0.07
Test accuracy: 96.67%

```

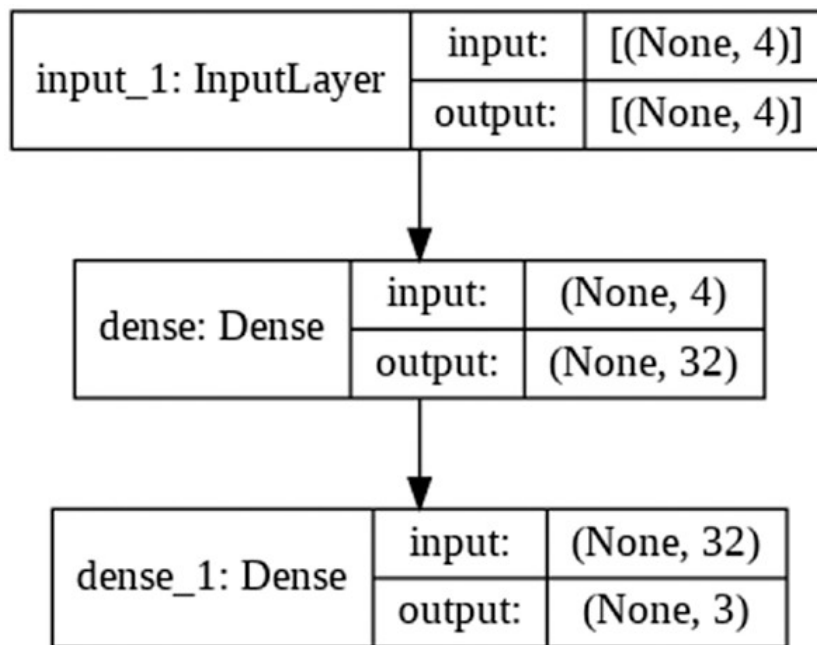


Figure 30-12. The graph of the model – produced with Keras

Model Visualization with Keras

With Keras, it is quite easy and straightforward to plot the metrics of the model to have a better graphical perspective as to how the model is performing for every training epoch. This view is also useful for dealing with issues of bias or variance of the model.

A callback function of the 'model.fit()' method returns the loss and evaluation score for each epoch. This information is stored in a variable and plotted.

In this example, we use the same Iris dataset model to illustrate visualization with Keras. The plots of the loss and accuracy of the model at each epoch are shown in Figure 30-13 and Figure 30-14, respectively.

```

!pip install -q tensorflow==2.0.0-beta0

# import packages
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import OneHotEncoder
  
```

```

# dataset url
train_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_training.csv"
test_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_test.csv"

# define column names
columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
'species']

# download and load the csv files
train_data = pd.read_csv(tf.keras.utils.get_file('iris_train.csv', train_
data_url),
                        skiprows=1, header=None, names=columns)

test_data = pd.read_csv(tf.keras.utils.get_file('iris_test.csv', test_data_url),
                        skiprows=1, header=None, names=columns)

# separate the features and targets
(X_train, y_train) = (train_data.iloc[:,0:-1], train_data.iloc[:,-1])
(X_test, y_test) = (test_data.iloc[:,0:-1], test_data.iloc[:,-1])

# apply one-hot encoding to targets
y_train=tf.keras.utils.to_categorical(y_train)
y_test=tf.keras.utils.to_categorical(y_test)

# create the functional model
def model_fn():
    # Model input
    model_input = tf.keras.layers.Input(shape=(4,))
    # Adds a densely-connected layer with 32 units to the model:
    x = tf.keras.layers.Dense(32, activation='relu')(model_input)
    # Add a softmax layer with 3 output units:
    predictions = tf.keras.layers.Dense(3, activation='softmax')(x)

    # the model
    model = tf.keras.Model(inputs=model_input,
                           outputs=predictions,
                           name='iris_model')

```

```

    # compile the model
    model.compile(optimizer='sgd',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model

# parameters
batch_size=50

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (X_train.values, y_train)).shuffle(len(X_train)).repeat().batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((X_test.values, y_test)).
batch(batch_size)

# build train model
model = model_fn()

# print train model summary
model.summary()

# train the model
history = model.fit(train_ds, epochs=10,
                   steps_per_epoch=100,
                   validation_data=test_ds)

# list metrics returned from callback function
history.history.keys()

# plot loss metric
plt.figure(1)
plt.plot(history.history['loss'], '--')
plt.plot(history.history['val_loss'], '--')
plt.title('Model loss per epoch: Training')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'evaluation'])
plt.show()

```

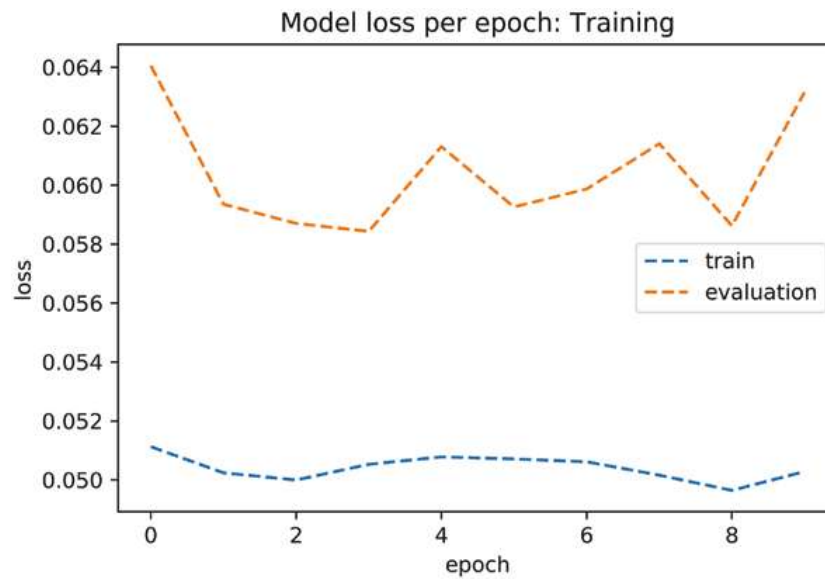


Figure 30-13. *Model loss per epoch*

```
# plot accuracy metric
plt.figure(2)
plt.plot(history.history['accuracy'], '--')
plt.plot(history.history['val_accuracy'], '--')
plt.title('Model accuracy per epoch: Training')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'evaluation'])
plt.show()
```

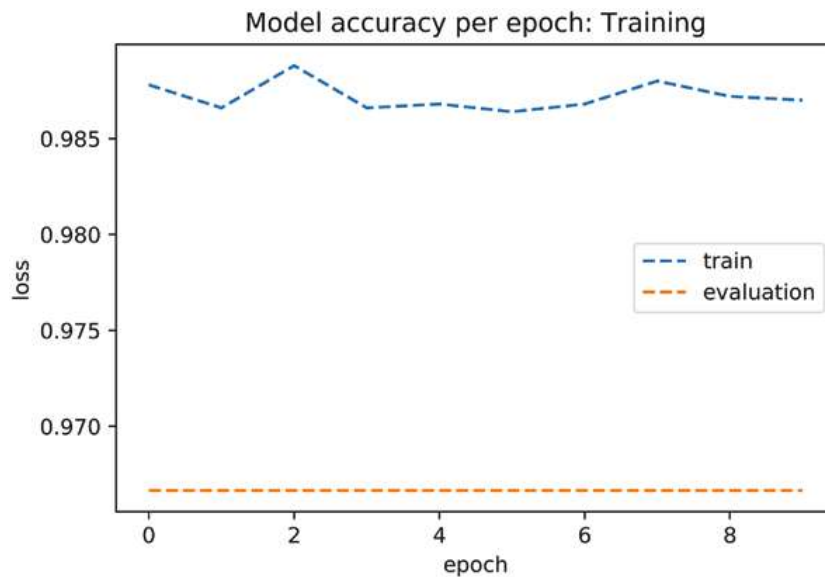


Figure 30-14. Model accuracy per epoch

TensorBoard with Keras

To visualize models with TensorBoard, attach a TensorBoard callback '**tf.keras.callbacks.TensorBoard()**' to the '**model.fit()**' method before training the model. The model graph, scalars, histograms, and other metrics are stored as event files in the log directory.

For this example, we modify the Iris model to use TensorBoard. The TensorBoard output is shown in Figure 30-15.

```
!pip install -q tensorflow==2.0.0-beta0

# import packages
import tensorflow as tf
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# load the TensorBoard notebook extension
%load_ext tensorboard

# dataset url
train_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_training.csv"
```



```

test_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_test.csv"

# define column names
columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
'species']

# download and load the csv files
train_data = pd.read_csv(tf.keras.utils.get_file('iris_train.csv',
train_data_url),
                        skiprows=1, header=None, names=columns)

test_data = pd.read_csv(tf.keras.utils.get_file('iris_test.csv', test_data_url),
                        skiprows=1, header=None, names=columns)

# separate the features and targets
(X_train, y_train) = (train_data.iloc[:,0:-1], train_data.iloc[:,-1])
(X_test, y_test) = (test_data.iloc[:,0:-1], test_data.iloc[:,-1])

# apply one-hot encoding to targets
y_train=tf.keras.utils.to_categorical(y_train)
y_test=tf.keras.utils.to_categorical(y_test)

# create the functional model
def model_fn():
    # Model input
    model_input = tf.keras.layers.Input(shape=(4,))
    # Adds a densely-connected layer with 32 units to the model:
    x = tf.keras.layers.Dense(32, activation='relu')(model_input)
    # Add a softmax layer with 3 output units:
    predictions = tf.keras.layers.Dense(3, activation='softmax')(x)

    # the model
    model = tf.keras.Model(inputs=model_input,
                           outputs=predictions,
                           name='iris_model')

```

```

# compile the model
model.compile(optimizer='sgd',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

return model

# parameters
batch_size=50

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (X_train.values, y_train)).shuffle(len(X_train)).repeat().batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((X_test.values, y_test)).
batch(batch_size)

# build train model
model = model_fn()

# print train model summary
model.summary()

# tensorboard
tensorboard = tf.keras.callbacks.TensorBoard(log_dir='./tmp/logs_iris_keras',
                                             histogram_freq=0, write_
                                             graph=True,
                                             write_images=True)

# assign callback
callbacks = [tensorboard]

# train the model
history = model.fit(train_ds, epochs=10,
                   steps_per_epoch=100,
                   validation_data=test_ds,
                   callbacks=callbacks)

```

```
# evaluate the model
score = model.evaluate(test_ds)
print('Test loss: {:.2f} \nTest accuracy: {:.2f}%'.format(score[0],
score[1]*100))

# execute the command to run TensorBoard
%tensorboard --logdir tmp/logs_iris_keras
```

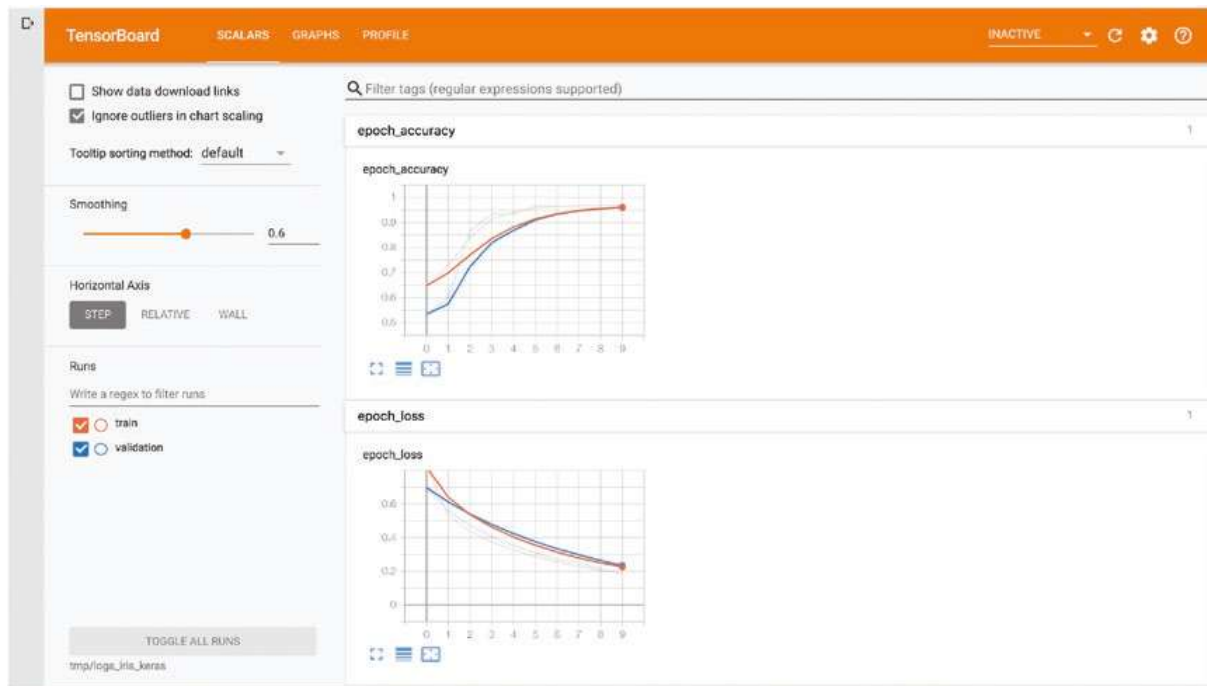


Figure 30-15. TensorBoard output of Iris model

Checkpointing to Select Best Models

Checkpointing makes it possible to save the weights of the neural network model when there is an increase in the validation accuracy metric. This is achieved in Keras using the `'tf.keras.callbacks.ModelCheckpoint()'`. The saved weights can then be loaded back into the model and used to make predictions. Using the Iris dataset, we'll build a model that saves the weights to file only when there is an improvement in the validation set performance. For completeness sake as we have done in the previous segments, we will produce this example within a complete code listing.

```
!pip install -q tensorflow==2.0.0-beta0
```

```

# import packages
import tensorflow as tf
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# dataset url
train_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_training.csv"
test_data_url = "https://storage.googleapis.com/download.tensorflow.org/
data/iris_test.csv"

# define column names
columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']

# download and load the csv files
train_data = pd.read_csv(tf.keras.utils.get_file('iris_train.csv', train_
data_url),
                        skiprows=1, header=None, names=columns)

test_data = pd.read_csv(tf.keras.utils.get_file('iris_test.csv', test_data_url),
                        skiprows=1, header=None, names=columns)

# separate the features and targets
(X_train, y_train) = (train_data.iloc[:,0:-1], train_data.iloc[:,-1])
(X_test, y_test) = (test_data.iloc[:,0:-1], test_data.iloc[:,-1])

# apply one-hot encoding to targets
y_train=tf.keras.utils.to_categorical(y_train)
y_test=tf.keras.utils.to_categorical(y_test)

# create the functional model
def model_fn():
    # Model input
    model_input = tf.keras.layers.Input(shape=(4,))
    # Adds a densely-connected layer with 32 units to the model:
    x = tf.keras.layers.Dense(32, activation='relu')(model_input)
    # Add a softmax layer with 3 output units:
    predictions = tf.keras.layers.Dense(3, activation='softmax')(x)

```

```

# the model
model = tf.keras.Model(inputs=model_input,
                        outputs=predictions,
                        name='iris_model')

# compile the model
model.compile(optimizer='sgd',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
return model

# parameters
batch_size=50

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (X_train.values, y_train)).shuffle(len(X_train)).repeat().batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((X_test.values, y_test)).
batch(batch_size)

# build train model
model = model_fn()

# print train model summary
model.summary()

# checkpointing
checkpoint = tf.keras.callbacks.ModelCheckpoint(
    './tmp/iris_weights.h5',
    monitor='val_accuracy',
    verbose=1,
    save_best_only=True,
    mode='max')

# assign callback
callbacks = [checkpoint]

```

```

# train the model
history = model.fit(train_ds, epochs=10,
                    steps_per_epoch=100,
                    validation_data=test_ds,
                    callbacks=callbacks)

# build evaluation model and upload saved weights
eval_model = model_fn()
eval_model.load_weights('./tmp/iris_weights.h5')

# evaluate the model
score = eval_model.evaluate(test_ds)
print('Test loss: {:.2f} \nTest accuracy: {:.2f}%'.format(score[0],
score[1]*100))

```

This chapter covered the foundation of working with TensorFlow 2.0 and its exciting features for developing machine learning models. Some of these new features include a more pythonic feel to model design and debugging, using `tf.function` to transform a Python method into high-performance TensorFlow graphs, using Keras as the core high-level API for model design, using `FeatureColumns` to parse data as input into Keras models, and the ease of training on distributed architectures and devices. The chapter also covered the principles of building models using the high-level Estimator API.

In the next chapters, we will take a deeper dive into deep neural networks and how they are implemented in TensorFlow with Keras. In TensorFlow 2.0, Keras is the de facto method for developing neural network.